

PaperLens — Snap-to-BI for Paper-Run Product Businesses

Working name. Turns photos of handwritten product lists, sales sheets, expense receipts, and raw-material counts into a live business-intelligence dashboard.

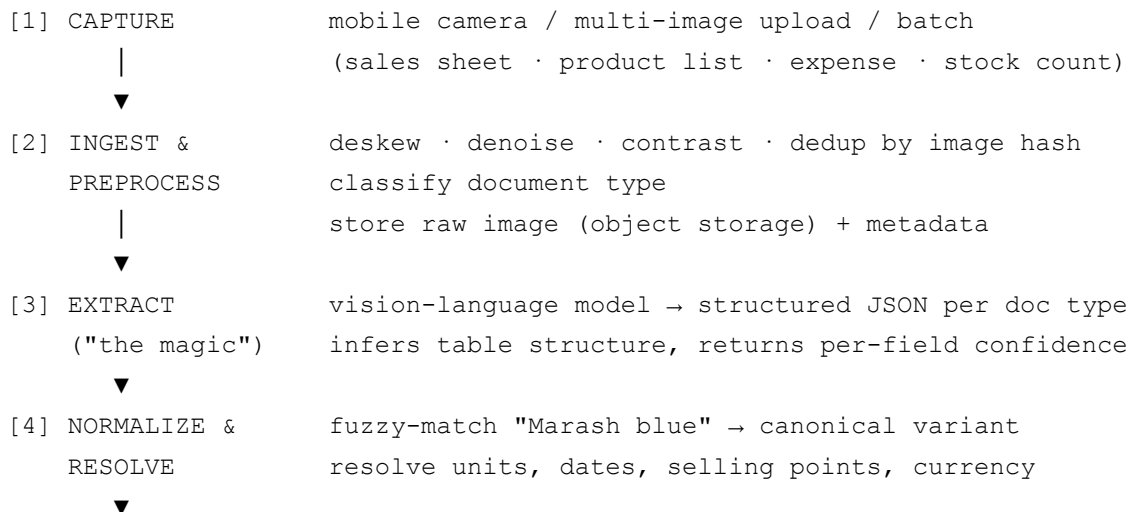
0. The one-liner

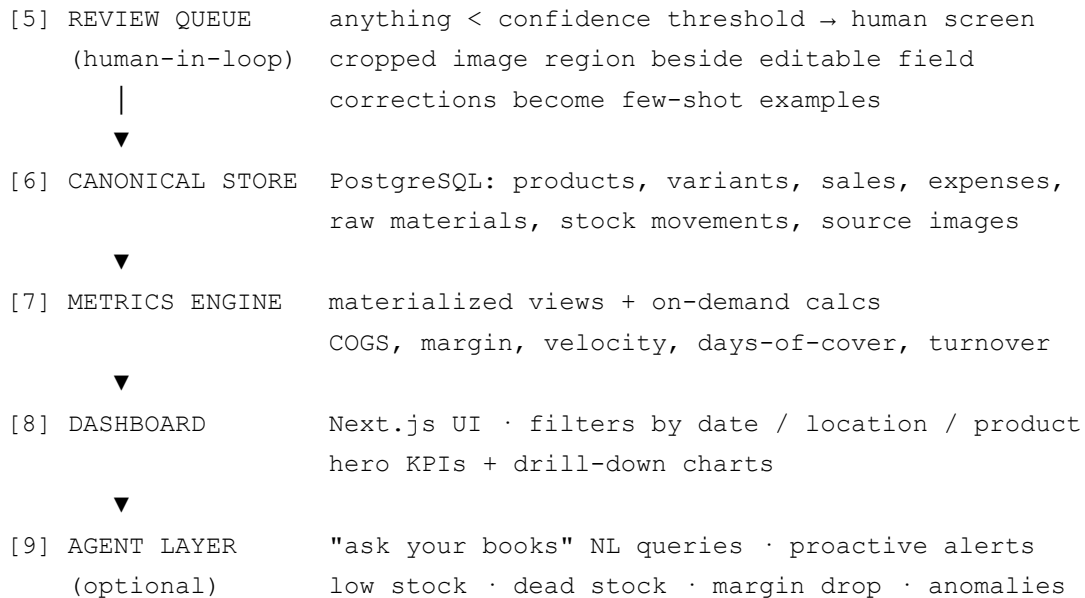
A shop owner who runs everything in a notebook photographs their pages. The system reads the handwriting, understands the structure ("this is a sales sheet for Tuesday at the Cascade kiosk"), maps it to a clean database, lets the owner fix anything the AI was unsure about, and renders profit, stock, and demand on a dashboard — without anyone ever typing a row into a spreadsheet.

The hard, valuable part is **not** the OCR. It is everything *after* the read: figuring out which scribbled product matches your real catalogue, what unit "12" is in, and turning a stack of pages into trustworthy numbers.

PART A — Architecture sketch

Nine layers, one direction of flow. Each layer is independently testable, which is what lets you ship a thin vertical slice first and fan out later.





The critical insight on sequencing: you cannot match a scribbled sale to a product if the product catalogue does not exist yet. So the *first* thing a new user uploads is their **product/price list** — that seeds the canonical catalogue. Only then do sales sheets have something to resolve against. Build and onboard in that order.

PART B — Data model (core entities)

Entity	Key fields	Why it matters
source_images	id, url, hash, doc_type, status, uploaded_at, processed_at	Every number traces back to a photo. Audit + dedup.
products	id, name, category, sku	The thing you sell.
variants	id, product_id, attributes (color/size), cost_price, sell_price	Per-variant pricing is where margin lives.
selling_points	id, name, location, type	Cascade, Megamall, etc. Performance varies wildly by location.
stock	variant_id, selling_point_id, qty_on_hand	Per-variant-per-location — matches a real multi-kiosk operation.
raw_materials	id, name, unit, qty_on_hand, reorder_threshold	Enamel, silver, thread — the inputs.

<code>bill_of_materials</code>	<code>variant_id, raw_material_id, qty_used</code>	Links a sale to material consumed → true COGS + auto-decrement.
<code>sales</code>	<code>id, variant_id, selling_point_id, date, qty, unit_price, total, payment_method, source_image_id</code>	The revenue spine.
<code>expenses</code>	<code>id, date, category, vendor, amount, currency, source_image_id</code>	Everything that isn't material cost.
<code>stock_movements</code>	<code>id, variant_id, qty_delta, reason, date, source_image_id</code>	In/out ledger so stock is reconstructable, not just a snapshot.
<code>review_items</code>	<code>id, source_image_id, field, raw_value, suggested_value, confidence, status</code>	The human-in-the-loop work queue.

Treat stock as a **movement ledger**, not a single number. A snapshot can't tell you *why* stock is wrong; a ledger can.

PART C — Full functional scope (what the system must do)

Capture

- Photograph or upload one or many pages at once; reorder before submitting.
- Works on a cheap phone, offline-tolerant (queue + sync when connected).
- Accept the four core document types: **product/price list, sales sheet, expense/receipt, raw-material count**.
- **Multilingual handwriting** including Armenian, mixed Armenian/English, and Armenian numerals — a first-class requirement, not an afterthought. This is also your hardest accuracy test, so design for it from day one.

Extract

- Detect document type automatically; let the user override.
- Return structured rows with a confidence score *per field*, not per page.
- Handle messy reality: crossed-out lines, totals in the margin, mixed units, two products on one line.

Normalize & resolve

- Fuzzy-match extracted product names to canonical variants; propose the top match with a score.
- Resolve dates ("Tues" + page context → ISO date), units (pcs vs grams), selling point, and currency (AMD/EUR).
- Flag genuinely new products for one-tap "add to catalogue."

Review (human-in-the-loop)

- A queue of only the uncertain fields — not a re-typing of the whole page.
- Side-by-side cropped image region + editable value.
- Every correction is stored and fed back as a few-shot example so accuracy climbs with use.

Store & reconcile

- Commit reviewed rows to the canonical store.
- Auto-decrement variant stock on a sale; auto-decrement raw materials via the bill of materials.
- Detect duplicates (same page photographed twice) and conflicting entries.

Metrics & dashboard

- Compute the KPI set in Part D over any date range, selling point, product, or category.
- Hero KPIs up top; drill-down on tap.
- "Data coverage" indicator: what % of recent days have been digitized, so the owner trusts the numbers.

Alerts & agent (optional, high-value)

- Reorder-now list when days-of-cover drops below threshold.
 - Dead-stock flag (no sales in N days).
 - Margin-drop and unusual-expense anomaly flags.
 - Natural-language queries: "what was my best seller at Megamall last month?"
-

PART D — The dashboard (metrics that actually matter)

Hero row (the 8 that earn the top of the screen)

Metric	Definition	Why it's a hero
Net profit (period)	Revenue – COGS – expenses	The number the owner actually cares about.
Revenue + trend	Σ sales, vs prior period	Direction of the business at a glance.
Gross margin %	(Revenue – COGS) / Revenue	Tells you if growth is healthy or hollow.
Units sold + best seller	Σ qty; top variant by units	What's moving.
Most profitable product	Top variant by <i>contribution margin</i>	Almost never the same as best seller — the killer insight.
Stock value	Σ (qty_on_hand $\times$ cost) at cost and at retail	Cash frozen in inventory.
Reorder-now count	# variants below days-of-cover threshold	Prevents stockouts of winners.
Expense burn	Σ expenses, vs prior period	The leak side of the ledger.

Full metric catalogue (grouped, for drill-downs)

Sales & demand

- Revenue by day / week / month, by selling point, by product, by category
- Units sold; average sale value
- Sell-through rate = sold / (sold + on hand)
- Sales velocity per variant (units/day)
- Best sellers and slow movers
- Location performance (revenue and margin per selling point)

Profitability

- Gross profit; gross margin %

- Contribution margin per variant (sell price – material cost – making cost)
- Most-profitable vs most-popular (plot them against each other)
- Net profit; break-even point

Inventory

- Stock on hand per variant per location
- Stock value at cost and at retail
- **Days of cover** = $\text{qty_on_hand} / \text{sales velocity}$ → the single best “do I reorder?” number
- Stockout-risk list
- Dead stock (no movement in N days) and its tied-up cash
- Inventory turnover ratio

Raw materials

- Material on hand vs consumption rate
- Days until each material runs out
- Material cost as % of revenue

Expenses & cash

- Expenses by category and trend
- Material vs non-material cost split
- Cash position / burn (if cash is tracked)

System health

- Data coverage (% of days digitized)
- Extraction accuracy and % of fields auto-approved (no human touch)

PART E — Recommended stack

- **Frontend / dashboard:** Next.js (App Router), Tailwind, a chart lib (Recharts/Chart.js)
- **Backend / API:** Next.js route handlers or a small Node service
- **DB:** PostgreSQL via Prisma

- **Image storage:** Cloudinary or S3-compatible object storage
- **Extraction:** a vision-LLM (Claude API with vision) prompted per doc type with a strict JSON schema + confidence fields
- **Hosting:** Railway
- **Queue:** a lightweight job queue for async extraction (so the phone gets an instant "received," processing happens in the background)

This deliberately mirrors a stack you can stand up fast and host cheaply.

PART F — Sprint plan

Two-week sprints, scoped for a small/solo technical team. The spine of the plan is **one vertical slice first** (photo → dashboard for sales only), then fan out. Each sprint ends in a demo you can actually show.

Sprint 0 — Foundations (1 week)

- **Goal:** repo, schema, deploy pipeline, image storage wired.
- **Scope:** Prisma schema for all core entities; Railway + Postgres + object storage live; auth; a stub upload endpoint that stores an image.
- **Done when:** you can upload a photo from your phone and see it stored with metadata.

Sprint 1 — Capture + catalogue seeding

- **Goal:** ingest the **product/price list** and build the canonical catalogue.
- **Scope:** capture UI (multi-image, mobile); preprocessing; extraction of a product list → `products / variants` ; basic review screen for this one doc type.
- **Done when:** you photograph your price list and your catalogue appears in the DB, corrections included.
- **Why first:** nothing downstream can resolve products until this exists.

Sprint 2 — Sales vertical slice (the proof)

- **Goal:** photo of a sales sheet → numbers on a screen, end to end.
- **Scope:** sales-sheet extraction → JSON with per-field confidence; entity resolution against the catalogue; auto-decrement stock; **one** real dashboard metric (revenue by

day).

- **Done when:** a Tuesday sales sheet becomes a revenue figure you trust, with low-confidence fields routed to review.
- **This is your demo-able MVP.** Everything after is breadth and depth.

Sprint 3 — Review queue + accuracy loop

- **Goal:** make the human-in-the-loop fast and self-improving.
- **Scope:** proper review UI (cropped region beside editable field); confidence thresholds; store corrections as few-shot examples; duplicate-page detection.
- **Done when:** correcting a page takes seconds, and re-uploading a fixed page-type shows measurably higher auto-approval.

Sprint 4 — Expenses + raw materials

- **Goal:** complete the cost side.
- **Scope:** expense/receipt extraction → `expenses` with categories; raw-material count extraction → `raw_materials` + movements; bill-of-materials so sales consume materials.
- **Done when:** net profit and material-on-hand are computable, not just revenue.

Sprint 5 — Metrics engine

- **Goal:** all KPIs in Part D, computed reliably.
- **Scope:** materialized views / scheduled jobs for COGS, margin, velocity, days-of-cover, turnover; date/location/product filters; data-coverage indicator.
- **Done when:** every metric in the catalogue returns correct numbers over arbitrary filters.

Sprint 6 — Dashboard UI

- **Goal:** the screen the owner opens every morning.
- **Scope:** hero KPI row; drill-down charts; location and product breakdowns; mobile-first layout.
- **Done when:** a non-technical owner can read profit, stock, and best/worst sellers without help.

Sprint 7 — Agent layer + polish (optional)

- **Goal:** proactive value.
- **Scope:** reorder-now and dead-stock alerts; margin/expense anomaly flags; natural-language “ask your books” query.
- **Done when:** the system tells the owner what to do, not just what happened.

PART G — Risks & mitigations

Risk	Mitigation
Handwriting accuracy, esp. Armenian / mixed script	Make multilingual a Sprint-1 test, not a late discovery; confidence thresholds + review queue catch the rest.
Cold-start: no catalogue to resolve against	Force product-list ingestion first (Sprint 1 before Sprint 2).
Faded receipts, crossed-out lines, two items per line	Per-field confidence + human review; never auto-commit low-confidence rows.
Unit ambiguity (pcs vs grams)	Define units per material/variant in the catalogue so extraction has a target to snap to.
Owner distrust of “magic numbers”	Data-coverage indicator + every number drills down to its source photo.
Over-scoping the MVP	The vertical slice (Sprints 0–2) is the only thing that has to ship to prove the concept.

PART H — Definition of done (system-level success)

- A user can go from a stack of paper pages to a trustworthy profit/stock/demand dashboard with **zero manual data entry** beyond correcting flagged fields.
- Target: **≥85% of fields auto-approved** without human touch after the accuracy loop has seen a few sessions of corrections.
- Every dashboard number traces back to a specific source photo.
- The owner can answer, in under 10 seconds: *Am I making money? What should I reorder? What’s my best product by profit, not just by sales?*

